BookLeaf Publishing

Author Support & Communication Portal

Full-Stack Developer Technical Interview & Architecture Guide

1. High-Level Architecture & Technical Choices

The BookLeaf Author Support Portal is built as a modular Client-Server architecture. The backend provides a RESTful API powered by Node.js, Express, and MongoDB, protected by JSON Web Tokens (JWT) stored in secure HTTP-only cookies. AI operations for classification and draft drafting run via OpenRouter (using OpenAI's gpt-4o-mini).

Why We Chose This Stack (Architecture Trade-offs):

• **Node.js & Express vs. Python (FastAPI/Django):** Node.js runs an asynchronous event-driven I/O model. Support ticket portals spend 90% of their operational time waiting on database queries and external AI API calls. Node.js handles thousands of concurrent I/O requests with minimal memory footprint compared to multi-threaded WSGI Python servers.

• **MongoDB vs. PostgreSQL:** Book publishing catalogs and author support queries deal with semi-structured and evolving domain entities (e.g. varying book stages, platform distribution arrays, and conversation replies). MongoDB's document model matches JSON natively, allowing nested arrays for 'responses' and 'internalNotes' without requiring complex multi-table JOINs.

• **HTTP-only Cookies vs. LocalStorage:** Storing JWTs in localStorage exposes the application to Cross-Site Scripting (XSS) token theft. By setting 'httpOnly: true' and 'sameSite: lax', the browser manages cookie transmission automatically and JavaScript running on the frontend can NEVER read or export the secret token.

• **Polling vs. WebSockets:** The assignment requires near-real-time updates without refreshing. Polling at 5-second intervals on the ticket page satisfies this with zero infrastructural overhead. WebSockets introduce stateful connection management, reconnect logic, socket heartbeat drops, and sticky session complexities when scaling behind reverse proxies.

• **GPT-4o-mini via OpenRouter vs. GPT-4o / Claude 3.5 Sonnet:** GPT-4o-mini costs ~95% less than frontier models while providing deterministic JSON output and ~800ms latency. For categorization and template-constrained customer support replies, mini models achieve identical accuracy at a fraction of the token cost.

2. Backend Directory Breakdown & Code Walkthrough

The server directory is structured with strict Separation of Concerns: routes handle URL matching, controllers execute business logic, models define database schemas, middlewares guard endpoints, and AI services isolate third-party LLM interactions.

A. Entry Point: server.js

• **Express 5 Setup:** Initializes the application, attaches CORS with 'credentials: true' (allowing secure cookies from localhost:5173), and sets up body-parser and cookie-parser.

• **Rate Limiting:** Applies 'express-rate-limit' restricting clients to 100 requests per 15-minute window. This protects authentication endpoints from brute-force password guessing and prevents AI quota depletion.

• **Server Initialization:** Awaits 'connectDB()' BEFORE calling 'app.listen()'. If the database is down, the server will not falsely report readiness.

B. Database Connection: config/db.js

Connects to MongoDB using Mongoose. We added 'serverSelectionTimeoutMS: 10000'. In production, if network connectivity drops or an IP is not whitelisted, the connection attempt fails fast and throws an informative error rather than hanging processes silently.

C. Data Models: models/users.js, books.js, tickets.js

• **User Model (users.js):** Enforces email uniqueness and lowercase normalization. Crucially, 'password: { type: String, select: false }' prevents password hashes from leaking in accidental 'User.find()' queries.

• **Book Model (books.js):** Indexes 'authorId' for fast lookup. Handles edge cases in the BookLeaf dataset: books still in production have null 'publication_date' and 'mrp', which our frontend and backend gracefully accommodate.

• **Ticket Model (tickets.js):** Contains strict enums for categories and priorities. Features separate arrays: 'responses' (author-visible communication) and 'internalNotes' (strictly restricted to administrators). Includes 'aiClassified: Boolean' and 'aiDraft: String' fields.

D. Security & Middlewares: middlewares/auth.js & adminsOnly.js

• **auth.js:** Reads 'req.cookies.bookleaf'. Verifies signature using process.env.SECRET_KEY. Attaches 'req.userId' and 'req.user' (containing role and authorId) to the incoming request object.

• **adminsOnly.js:** Inspects 'req.user.role'. If role !== 'admin', rejects immediately with HTTP 403 Forbidden.

E. AI Engine: AI/aiService.js & AI/knowledgeBase.js

• **Sliced Knowledge Base (knowledgeBase.js):** Contains a dictionary of policies mapped by category. Instead of sending the full company policies on every prompt, we ONLY inject the policy corresponding to the ticket's category. This saves ~70% token usage per request!

• **Non-blocking AI Classification (classifyTicket):** Called on ticket creation with 60 max_tokens. It requests strict JSON: '{"category": "...", "priority": "..."}'. The response is validated against whitelisted enums before saving.

• **AI Draft Reply (draftReply):** Generates grounded support responses (max 400 tokens). It injects the author's book data, specific policy SLAs (48h for ISBN, 5-7 days for reprints, 80/20 royalty split), and BookLeaf's empathetic tone.

F. Core Controllers: controllers/ticketController.js & aiController.js

• **createTicket:** Saves ticket instantly with defaults. Then fires 'classifyTicket()' asynchronously in the background. If the AI is slow or fails, the author never waits!

• **getMyTickets & getTicketById:** Applies 'AUTHOR_HIDDEN_FIELDS = "-internalNotes -aiDraft"'. When an author requests their ticket, internal notes and AI drafts are completely stripped from the response.

• **getAllTickets:** Sorts using an in-memory priority rank: Critical (0), High (1), Medium (2), Low (3), followed by createdAt ascending. This ensures the most urgent and oldest unresolved tickets always appear at the very top of the admin queue.

• **replyToTicket:** Appends admin message to responses array. Automatically transitions ticket status from 'Open' to 'In Progress' on the first reply.

• **getDraft (aiController.js):** Draft caching: checks if 'ticket.aiDraft' already exists. If yes, returns it instantly without making an external API call! Only calls OpenRouter if no draft exists or if '?regenerate=true' is passed.

3. Top 15 Technical Interview Questions & Perfect Answers

Q: How did you handle security and prevent authors from seeing internal notes or other authors' tickets?

Answer: We implemented Role-Based Access Control (RBAC) at both route and query levels. First, authFn middleware verifies the JWT cookie and attaches req.user. In getMyTickets and getTicketById, we sanitize projections by excluding '-internalNotes -aiDraft'. Furthermore, if an author requests a ticket by ID that does not match their req.user.authorId, the controller returns a 403 Forbidden.

Q: What happens if the AI API (OpenRouter) is down, rate-limited, or throws an error?

Answer: Our architecture implements graceful degradation. When creating a ticket, AI auto-classification is executed in a non-blocking Promise (.then().catch()). If OpenRouter fails, the catch block logs the error, and the ticket retains its default valid category ('General Inquiry') and priority ('Medium'). For draft replies, if the AI fails, the endpoint returns HTTP 503 with an informative message, allowing support admins to write responses manually.

Q: How did you optimize AI token usage and control operational API costs?

Answer: We applied three concrete optimizations: (1) Sliced Knowledge Base: Only the specific policy section relevant to the ticket category is injected into the prompt rather than the entire employee handbook, saving ~70% prompt tokens. (2) Strict Max Tokens: We capped classification at 60 tokens and response drafts at 400 tokens. (3) Draft Caching: Once generated, drafts are saved in MongoDB (ticket.aiDraft); opening a ticket multiple times reuses the saved draft at $0 cost unless '?regenerate=true' is explicitly requested.

Q: Why did you use HTTP-only cookies instead of LocalStorage for authentication?

Answer: LocalStorage is accessible to any JavaScript running on the page. In the event of an XSS vulnerability (e.g. from a compromised third-party script or user-injected markup), tokens in localStorage can be stolen. HTTP-only cookies cannot be accessed via document.cookie, mitigating token extraction.

Q: Why did you choose HTTP polling over WebSockets for real-time ticket updates?

Answer: For this scale, HTTP polling (5s for authors, 10s for admin queue) satisfies the requirement of updating without page refresh while keeping the architecture stateless. WebSockets introduce persistent TCP connections, reconnection logic, ping/pong heartbeats, and require sticky sessions or Redis pub/sub when clustering across multiple server instances.

Q: Why did you mark the password field with 'select: false' in the Mongoose schema?

Answer: In MongoDB/Mongoose, querying a user document normally returns all fields by default. If a developer runs User.findOne() or User.find(), password hashes could accidentally be serialized into JSON responses or logs. Setting 'select: false' excludes the password field from all queries unless explicitly requested using .select('+password').

Q: How does your ticket sorting algorithm prioritize unresolved queries in the Admin Queue?

Answer: In getAllTickets, we map priorities to numerical ranks: Critical: 0, High: 1, Medium: 2, Low: 3. We then sort using a dual-condition comparator: PRIORITY_RANK[a.priority] - PRIORITY_RANK[b.priority] || a.createdAt - b.createdAt. This guarantees that all Critical tickets appear first, and within the same priority level, the oldest tickets appear at the top.

Q: What is the net profit formula BookLeaf uses for author royalties, and how does your AI draft reflect it?

Answer: Net profit = MRP minus printing cost, platform commission (Amazon/Flipkart), and shipping charges. The author receives 80% of net profit, and BookLeaf receives 20%. Our AI draft prompt is specifically instructed with this formula so that when an author queries why their payout seems lower than total retail copies sold, the AI transparently explains the calculation without defensive language.

Q: How did you handle the edge cases present in the sample dataset (bookleaf_sample_data.json)?

Answer: The dataset contained books in production (e.g. 'Midnight in Mysore' and 'Raising Roots') where publication_date and mrp are null, and sales/royalties are 0. In Books.jsx, we added conditional rendering so these books display 'In Production' badges and a message stating royalty details will appear once live, avoiding unsightly NaN or empty values.

Q: Why did you wrap the AI classification response in a safety check?

Answer: LLMs can occasionally hallucinate categories outside the allowed schema or wrap JSON in Markdown fences (```json). Our classifyTicket function strips Markdown backticks, parses the JSON, and explicitly verifies: if (!CATEGORIES.includes(result.category) || !PRIORITIES.includes(result.priority)), it throws an error and falls back to safe system defaults.

Q: How would you scale this application to support 100,000 authors?

Answer: To scale to enterprise volume: (1) Background Job Queues: Move AI classification and transactional emails to BullMQ with Redis workers. (2) Database Optimization: Add compound indexes on { authorId: 1, status: 1 } and { priority: 1, createdAt: 1 }. (3) Caching: Cache author book lists in Redis with TTL. (4) Cloud Storage: Connect attachment uploads to AWS S3 with pre-signed URLs. (5) Vector DB: Implement RAG using Pinecone for large-scale knowledge base search.